

Hunter College
Department of Computer Science

CSCI 39548: Practical Web Development
Summer 2026

Assignment 2 — Expense Tracker (JavaScript + DOM + Async)

Instructor: *Sourya Saha* (sourya.saha24@login.cuny.edu)

Weight	10% of course grade
Released	Monday, June 8, 2026
Due	Monday, June 15, 2026, by 6:40 PM (before class)
Submission	Brightspace — paste a link to your public GitHub repository + a link to your merged pull request
Type	Individual

1 Overview

Build a small **personal expense tracker** as a single-page web app using **plain JavaScript** — no frameworks. The app lets a user add expenses, list them, delete them, see running totals, and have their data survive a page refresh.

This assignment is the JS counterpart to A1: A1 was about *structure and style*, A2 is about *behavior* — manipulating the DOM, handling events, working with arrays, persisting state, and making a network request.

Build it as we teach it. The assignment is released the same day we start JavaScript so you can grow your solution alongside the lectures. A suggested rhythm:

- **After the first JS class** — Scaffold the HTML/CSS, model the expense as a JS object, get an in-memory array, and use array methods to compute totals.
- **After the second JS class** — Wire up the DOM render, form events, `localStorage`, and the `fetch` call.
- **By Mon Jun 15** — Polish, validate, handle empty/error states, push the final PR.

*Note: No frameworks, no build tools. Write plain HTML, CSS, and JavaScript. Do **not** use React, jQuery, Tailwind, Bootstrap, Vite, or npm packages. Open `index.html` in the browser and the app should run.*

2 Learning Objectives

- **CO3** — Write JavaScript using variables, control flow, functions, and array methods to model and transform application state.
- **CO4** — Manipulate the DOM in response to user events. Persist state with `localStorage`. Consume an HTTP API with `fetch` and `async/await`.

3 Part 1 — The Expense Tracker (Core Features)

Build a one-page app (`index.html` + `styles.css` + `script.js`) that supports **all** of the following:

- **Add an expense** via a form with four labeled inputs: **description** (text), **amount** (number ≥ 0), **category** (a `<select>` with at least 4 categories — e.g. *Food*, *Transport*, *Entertainment*, *Other*), and **date** (`<input type="date">`, defaulting to today).

- **Validate the form** in JS before adding: description non-empty, amount > 0 , date present. Show a clear inline error and do **not** add the expense if invalid.
- **Render the list** of expenses in the DOM — each row shows description, amount (formatted as currency, e.g. \$12.50), category, and date.
- **Delete** any expense via a button on its row. The list re-renders.
- **Running totals**, recomputed on every change:
 - Overall total (sum of all amounts).
 - **Per-category totals** (use `reduce` or equivalent).
 - **Count** of expenses currently shown.
- **Filter by category** — a `<select>` (or set of buttons) that shows only matching rows. An “All” option resets the filter. Totals shown reflect the **filtered** view.
- **Sort** the list by **date** or by **amount**, ascending or descending, controlled by the user (a `<select>` or buttons).
- **Empty state** — when there are no expenses (or the filter matches none), show a friendly message instead of an empty list.

3.1 Code Quality Requirements

- State lives in a **JS array of objects**. The DOM is **rendered from that array**, not mutated ad-hoc. (Recommended pattern: a single `render()` function called after every state change.)
- Use `const` / `let` — **no var**.
- Use **array methods** (`map`, `filter`, `reduce`, `sort`) where they fit naturally — not hand-rolled `for` loops for everything.
- Event listeners attached with `addEventListener` — **no inline** `onclick="..."` in the HTML.

4 Part 2 — Persistence (localStorage)

- Save the expense list to `localStorage` after every change.
- On page load, read it back so the user’s data **survives a refresh**.
- If `localStorage` is empty or its contents can’t be parsed, fall back to an empty list (don’t crash).

5 Part 3 — Async (fetch + async/await)

Add a “**Convert to EUR**” (or any non-USD currency) button next to the overall total. Clicking it:

1. Calls a **live exchange-rate API** with `fetch` inside an `async` function using `await`.
2. While the request is in flight, shows a **loading state** (e.g. button disabled, text “Converting...”).
3. On success, displays the converted total alongside the USD total.
4. On error (network failure, bad response), shows a clear error message — **do not** silently swallow the error.

You may use any free, no-key exchange-rate endpoint. Two known-good options:

- <https://open.er-api.com/v6/latest/USD>
- <https://api.exchangerate-api.com/v4/latest/USD>

*Note: Both return a JSON object with a **rates** field — read the rate for your target currency from there. Test it in the browser address bar first to see the shape.*

6 Submission

Submit on **Brightspace**. Paste **two** links:

1. Your public **GitHub** repository URL — e.g.
`https://github.com/yourusername/expense-tracker`

2. Your merged pull request URL — e.g.

`https://github.com/yourusername/expense-tracker/pull/1`

The grader will **clone the repo and open index.html in a browser** — make sure the app runs that way. Do **not** submit a `.zip` and do **not** rely on a build step.

Note: Verify your links work in a private/incognito window before submitting.

7 Git/GitHub Requirements

Same workflow as A1 — this is not new:

- Public GitHub repository with a `README.md` and a `.gitignore`.
- Work on a **feature branch**, not directly on `main`.
- **Several meaningful commits** with clear messages.
- **At least one pull request** merged into `main`.

A clean workflow is part of the rubric. A working app committed straight to `main` in one giant commit called “done” will lose points.

8 Grading Rubric (100 pts)

Criterion	Pts
Core features work — add, delete, render, filter, sort, totals all correct	30
Validation & empty state — invalid input rejected with feedback; empty list handled	10
State + render pattern — single source of truth in JS, DOM rendered from state	10
Array methods used idiomatically — <code>map</code> / <code>filter</code> / <code>reduce</code> / <code>sort</code> where appropriate	10
localStorage persistence — survives refresh; handles missing/-corrupt data	10
fetch + async/await — request, loading state, success render, error path	15
Git workflow — feature branch, PR, merge, clear commits, README, <code>.gitignore</code>	10
Code readability — naming, structure, no dead code, no <code>console.log</code> spam left in	5

9 Notes

- **Late policy:** see the syllabus. Up to one week late (with prior contact) at up to -20% per day.
- **Use of AI:** allowed and encouraged, but you must be able to **explain every line you submit**. Don't ship JS you can't read.
- **No frameworks, no build tools.** If the grader has to run `npm install`, it's the wrong stack for this assignment.
- **Browser console = your friend.** `console.log`, breakpoints, the Network tab, and Application → Local Storage are all fair game while you build. Just remove leftover `console.log` calls before you submit.

- **What this prepares you for:** A3 (React) replaces your manual `render()` with React's `render`. The final project replaces `localStorage` with a real database. The patterns you build here — state, render, events, fetch — are exactly the patterns we'll keep using.

End of Assignment 2